

Colas de prioridad Montículos (Heaps)

Aun cuando los trabajos enviados a una impresora en línea se suelen colocar en una cola, esto puede no ser siempre lo mejor. Por ejemplo, un trabajo puede ser de particular importancia, de modo que es deseable permitir que se lleve a cabo tan pronto como la impresora esté disponible. A la inversa, si al quedar disponible la impresora hay varios trabajos de una página y uno de cien, puede ser razonable dejar el trabajo grande para el final, aun cuando no sea el último enviado. (Desafortunadamente, la mayoría de los sistemas no hacen esto)

De manera similar, en un entorno multiusuario, el planificador del sistema operativo debe decidir entre varios procesos cuál ejecutar. En general se permite que un proceso se ejecute sólo durante un tiempo fijo. Un esquema (algoritmo) usa una cola donde al inicio los trabajos se colocan al final de la cola y el planificador tomará repetidamente el primer trabajo de la cola, lo ejecutará hasta que termine o alcance su límite de tiempo, y lo colocará al final de la cola si no termina. Por lo general, esta estrategia no es adecuada porque trabajos muy cortos parecerán lentos debido al tiempo de espera para su ejecución. Por lo regular, es importante que los trabajos cortos terminen tan pronto como sea posible, de modo que estos trabajos deberían tener preferencia sobre los trabajos que ya han estado en ejecución. Además, algunos trabajos no cortos son muy importantes y deben también tener preferencia.

Esta aplicación particular parece requerir una clase especial de cola, denominada *cola de prioridad* o **montículo**.

Un montículo es una estructura de datos similar a un árbol binario de búsqueda pero ordenado de distinta forma por prioridades y además se representa siempre como un árbol binario completo representado como un arreglo.

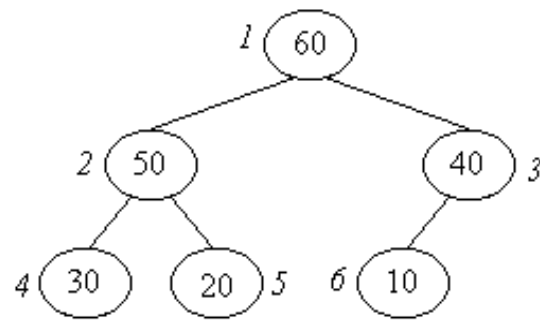
Definición:

Un montículo es un árbol binario completo tal que:

- a) es vacío, o
- b) el valor de la prioridad en la raíz es mayor o igual que la prioridad de cualquiera de sus hijos, y
- c) ambos subárboles son montículos

Nota: en el inciso b) la palabra mayor puede substituirse por menor.

Ejemplo:



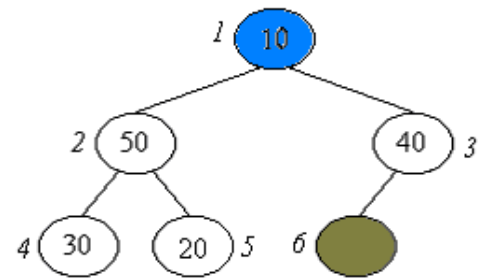
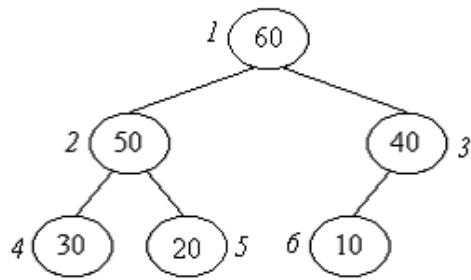
que en un arreglo aparecería representado como:

1	60
2	50
3	40
4	30
5	20
6	10
7	
8	
9	
	...
max	

Los montículos pueden utilizarse para implementar una cola de prioridades. La estructura de datos puede definirse de la siguiente manera:

En general, cuando la llave en la raíz es eliminada, el nodo que la contiene no puede eliminarse ya que quedarían dos hijos sin padre, por lo tanto necesitamos un método que reestablezca la propiedad de montículo.

Suponga que cualquier hoja es eliminada y la llave es transferida a la raíz. Claramente la propiedad de **heap** puede ser violada. Por ejemplo, considere el árbol anterior (representado en la siguiente figura del lado izquierdo), donde se desea eliminar la raíz, sustituyéndola por alguna hoja (representado en la siguiente figura del lado derecho)

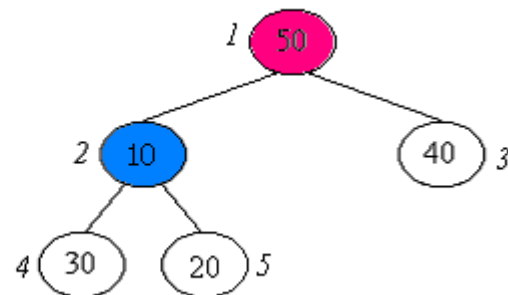
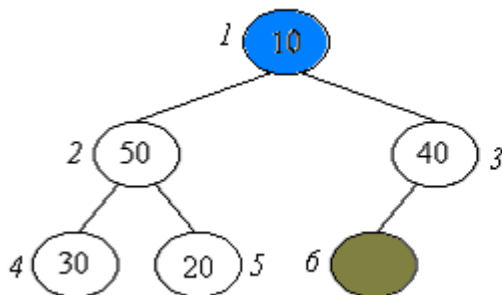


El problema aquí es que el elemento de la raíz no satisface la propiedad de montículo. Esta estructura es llamada un semimontículo y ahora se puede considerar la estrategia de transformar el semimontículo en un montículo.

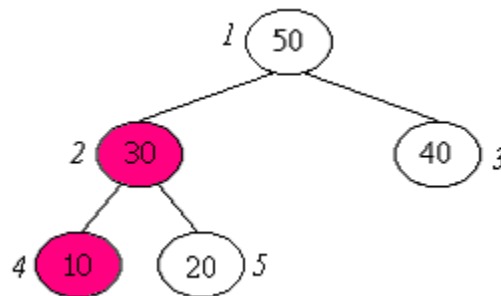
La única violación de la propiedad de montículo antes de la restauración es que la llave K en el árbol sea muy pequeña. Si es así, se puede cambiar a K con la llave de su hijo más grande, y entonces la nueva llave de la raíz será más grande que las llaves de sus hijos. Ahora la única posible violación de la propiedad de montículo es de nuevo que K sea muy pequeña y deba moverse un nivel hacia abajo. Esto puede manejarse a través de llamadas recursivas y eventualmente se obtendrá la propiedad de montículo hasta que quizá K no tenga hijos.

Considere lo anterior de manera gráfica:

El árbol anterior cuya raíz es 10 y tiene como hijos al 50 y al 40, para restablecer la propiedad de montículo y como 50 es el mayor se intercambia su valor con el 10:



Con este cambio ahora el semimontículo se encuentra con raíz en el nodo numerado con el 2, porque 10 es menor que 30 y 20, lo cual nos lleva a intercambiar el valor de 30 hacia la raíz:



Como puede observarse en la representación de almacenamiento contiguo es fácil definir funciones que regresen al hijo izquierdo y al hijo derecho. Cada función toma como argumento un índice del arreglo.

Insertar:

El algoritmo de inserción trabaja en forma contraria a la de borrar. En este caso, una llave K desea insertarse en un montículo. Esta puede insertarse en cualquier localidad disponible del árbol, de esta forma, la única violación que puede ocurrir es que K sea más grande que su padre y que necesite moverse un nivel hacia arriba en el árbol.

En la representación de almacenamiento secuencial, simplemente se incrementa el tamaño del montículo en uno y se incluye la nueva llave al final de las n llaves que forman el árbol y verificamos que se cumpla la propiedad de montículo. En el caso que no se cumpla será necesario intercambiar su valor con el de su padre hasta que su padre sea mayor que él.

Suponga ahora, que se tiene un árbol binario completo cuyas llaves no cumplen la propiedad de montículo ¿cómo convertirlo en un montículo?

Una forma de resolver el problema consiste en convertir cada subárbol en un montículo, así se tiene un árbol con raíz y dos subárboles que son montículos, el árbol completo será un montículo si se intercambia la llave del nodo raíz con la llave mayor de sus dos hijos: